📚 Manual Completo de Sistemas Operativos

🖥️ 1. Introdução aos Sistemas Operativos

O que é um Sistema Operativo (SO)?

Embora não exista uma definição universalmente consensual (a Microsoft, por exemplo, chegou a argumentar que o browser fazia parte do SO), a definição técnica mais aceite é que o SO é um programa de controlo.

Ele serve como intermediário entre o utilizador e o hardware, com dois objetivos principais:

1. **Abstração:** Tornar o uso do computador mais simples, escondendo a complexidade do hardware.
2. **Gestão de Recursos:** Permitir a execução eficiente de programas, gerindo processador, memória e discos.

Conceitos Fundamentais

Para entender um SO, é necessário dominar estes termos:

- **Processo:** É a unidade de execução. Um programa é passivo (está no disco), um processo é ativo (está a correr na memória).
- **Memória Virtual:** Uma técnica que "engana" os programas, simulando que existe mais memória RAM do que a física. O SO usa parte do disco rígido para armazenar dados que não cabem na RAM.
- **Sistema de Ficheiros:** A forma como os dados são organizados no disco (pastas e ficheiros).
- **Drivers (Periféricos):** Pequenos programas que ensinam o SO a comunicar com hardware específico (rato, impressora, placa gráfica).
- **Máquina Virtual:** Software que emula um computador completo, permitindo rodar um SO dentro de outro.

Evolução Histórica

A evolução dos SOs reflete a busca por eficiência:

1. **Primeiros Computadores (1950):** Eram gigantes e usados por cartões perfurados. Não havia SO real, apenas monitores de controlo básicos.
2. **Batch Processing (Lotes):** Para evitar que a CPU ficasse parada enquanto o operador trocava cartões, os trabalhos eram agrupados em lotes. Introduziu-se o **Spooling** (usar o disco como buffer temporário), permitindo

que a leitura de entrada e a impressão de saída acontecessem enquanto a CPU processava outra tarefa.

3. **Multi-programação:** A grande revolução. Vários programas residem na memória ao mesmo tempo. Quando um programa pausa (ex: à espera de ler um ficheiro), a CPU salta para outro. Isso maximiza o uso do processador.

Tipos de Sistemas Modernos:

- **Interativos:** O utilizador interage em tempo real (ex: Windows, Linux).
- **Paralelos:** Um computador com múltiplos processadores/núcleos a trabalhar juntos.
- **Distribuídos:** Vários computadores separados a trabalhar em conjunto via rede (ex: a Cloud).

⚙️ 2. Processos

Definição e Estrutura

Um processo é um **programa em execução**. Ele não é apenas o código, mas também o seu "contexto" atual. Um processo contém:

- **Código:** As instruções do programa.
- **Stack:** Memória temporária para variáveis locais e chamadas de função.
- **Program Counter (PC):** Um ponteiro que diz "qual é a próxima instrução a executar".
- **PCB (Process Control Block):** É a "ficha técnica" do processo guardada pelo SO. Contém o estado atual, registos da CPU, memória alocada e ficheiros abertos.

Ciclo de Vida (Estados)

Um processo passa por vários estados:

1. **Novo:** Acabou de ser criado.
2. **Pronto (Ready):** Está na memória e pronto a correr, só à espera que a CPU fique livre.
3. **Execução (Running):** Está efetivamente a usar a CPU.
4. **Espera (Waiting):** Parou porque precisa de algo (ex: input do utilizador ou leitura de disco).

5. **Terminado:** Acabou a execução.

Escalonamento (Scheduling)

Como a CPU só executa um processo de cada vez (por núcleo), o SO precisa de um "árbitro" (o **Dispatcher**) para decidir quem corre a seguir.

Algoritmos de Escalonamento:

- **FCFS (First-Come, First-Served):** O primeiro a chegar é o primeiro a ser servido. É justo, mas ineficiente se um processo lento bloquear os rápidos.
- **SJF (Shortest Job First):** Escolhe o processo mais curto. É o mais rápido em média, mas difícil de implementar porque o SO raramente sabe quanto tempo um processo vai demorar.
- **Prioridade:** Executa o mais importante. O perigo é o **Starvation** (processos de baixa prioridade nunca correm). A solução é o **Aging** (aumentar gradualmente a prioridade de quem espera há muito tempo).
- **Round-Robin (RR):** Cada processo recebe uma fatia de tempo (**Quantum**). Se não acabar nesse tempo, volta para o fim da fila. É o sistema usado em computadores pessoais para dar a sensação de que tudo corre ao mesmo tempo.

🔬 3. Comunicação entre Processos (IPC)

Processos frequentemente precisam de colaborar (partilhar dados). Existem duas formas principais de o fazer:

1. **Memória Partilhada:** O SO cria uma zona de RAM que ambos os processos podem ler e escrever. É muito rápido, mas complexo de sincronizar (para não escreverem ao mesmo tempo).
2. **Troca de Mensagens:** Os processos enviam dados através do SO (enviar() e receber()). É mais seguro e funciona mesmo entre computadores diferentes (rede).

Modos de Comunicação:

- **Síncrona (Bloqueante):** O remetente envia a mensagem e **fica parado** até que o recetor a receba. Ocorre um "Rendezvous" (encontro).
- **Assíncrona (Não Bloqueante):** O remetente envia e continua a trabalhar, sem saber se a mensagem já chegou.

Buffering (Filas de Mensagens):

Se o recetor não estiver pronto, onde fica a mensagem?

- **Capacidade Zero:** Não há fila. O remetente tem de esperar (obriga a sincronismo).
- **Capacidade Limitada:** Há uma fila finita. Se encher, o remetente bloqueia.
- **Capacidade Ilimitada:** O remetente nunca bloqueia (teoricamente).

🚦 4. Sincronização e Concorrência

Quando múltiplos processos tentam alterar os mesmos dados ao mesmo tempo, ocorre uma **Race Condition** (Condição de Corrida/Competição), onde o resultado final é imprevisível e geralmente incorreto.

A Secção Crítica

Para evitar erros, define-se a **Secção Crítica (SC)**: a parte do código onde o recurso partilhado é acedido. Para proteger a SC, devem cumprir-se 3 requisitos:

1. **Exclusão Mútua:** Se um processo está na SC, mais nenhum pode entrar.
2. **Progresso:** Se a SC está livre e alguém quer entrar, não pode ser impedido indefinidamente.
3. **Espera Limitada:** Um processo não pode ficar eternamente na fila para entrar na SC (evita starvation).

Ferramentas de Sincronização

- **Semáforos:** São variáveis especiais geridas pelo SO que funcionam como sinais de trânsito. Têm duas operações: wait() (diminui o contador e bloqueia se for 0) e signal() (aumenta o contador e acorda quem espera). São usados para garantir exclusão mútua.

Problemas Clássicos

- **Jantar dos Filósofos:** 5 filósofos, 5 garfos. Para comer precisam de 2 garfos. Se todos pegarem no garfo da esquerda ao mesmo tempo, ninguém come (**Deadlock**).
- **Leitores-Escritores:** Várias pessoas podem ler um ficheiro ao mesmo tempo, mas se alguém quiser escrever, tem de ter acesso exclusivo.

🔒 5. Deadlocks (Impasses)

Um Deadlock é uma situação onde um conjunto de processos fica bloqueado para sempre, pois cada um está à espera de um recurso que o outro tem.

Exemplo: O processo A tem a impressora e quer o scanner. O processo B tem o scanner e quer a impressora. Nenhum avança.

As 4 Condições Necessárias

Para haver Deadlock, **todas** estas condições têm de ser verdadeiras simultaneamente:

1. **Exclusão Mútua:** Recurso só pode ser usado por um de cada vez.
2. **Retenção e Espera:** Quem tem recursos pode pedir mais.
3. **Não Preempção:** Não se pode "roubar" recursos a um processo à força.
4. **Espera Circular:** A espera B, B espera C, C espera A.

Como lidar com Deadlocks?

1. **Prevenção:** Desenhar o sistema de forma a que pelo menos uma das 4 condições seja impossível (ex: obrigar a pedir todos os recursos de uma vez).
2. **Evitação:** O sistema analisa cada pedido. Se um pedido puder levar a um estado inseguro (risco de deadlock), o SO nega o recurso. O algoritmo mais famoso é o **Algoritmo do Banqueiro**.
3. **Deteção e Recuperação:** Deixar acontecer, detetar o ciclo e resolver (geralmente matando um processo ou tirando-lhe recursos).

🧠 6. Gestão de Memória

O SO tem de gerir a RAM para acomodar múltiplos processos.

Técnicas de Alocação

- **Partições Fixas:** A memória é dividida em blocos de tamanho fixo. Simples, mas causa **Fragmentação Interna** (se o processo for menor que o bloco, sobra espaço desperdiçado dentro do bloco).
- **Partições Variáveis:** O SO dá exatamente o espaço que o processo precisa. Evita fragmentação interna, mas cria **Fragmentação Externa** (buracos de memória livre espalhados que são demasiado pequenos para serem usados).
- **Algoritmo Buddy:** Compromisso inteligente. A memória é dividida em potências de 2 (ex: 256KB divide em dois de 128KB). Facilita juntar blocos vizinhos livres, mas ainda tem alguma fragmentação interna.

Algoritmos de Ajuste (Fitting)

Quando chega um processo, onde o colocamos?

- **First Fit:** No primeiro buraco onde caiba (rápido).
- **Best Fit:** No buraco onde caiba mais à justa (deixa buracos pequenos inúteis).
- **Worst Fit:** No maior buraco disponível (deixa sobras grandes, potencialmente úteis).

📖 7. Memória Virtual e Paginação

A memória virtual permite correr programas maiores do que a RAM física, usando o disco como extensão. A técnica mais comum é o **Paging (Paginação)**.

Como funciona o Paging?

1. A memória é dividida em pequenos blocos chamados **Páginas** (Virtual) e **Frames/Quadros** (Física/RAM).
2. A **MMU (Memory Management Unit)** é o hardware que traduz o "Endereço Virtual" do programa para o "Endereço Físico" real na RAM.
3. **Page Fault:** Se o programa tenta aceder a uma página que não está na RAM, a CPU gera uma interrupção. O SO vai buscar essa página ao disco, carrega-a na RAM e o programa continua como se nada tivesse acontecido.

🛡️ 8. Segurança e Proteção

A segurança em SOs visa proteger os dados e os recursos do sistema.

Modos de Operação da CPU

Para evitar que um utilizador destrua o sistema, a CPU alterna entre dois modos:

1. **Modo Kernel (Supervisor):** Acesso total a todo o hardware e instruções. É onde o SO corre.
2. **Modo Utilizador:** Acesso restrito. Não pode aceder diretamente ao hardware.

Chamadas ao Sistema (System Calls): Quando um programa precisa de ler um ficheiro ou usar a rede, não o faz diretamente. Ele pede ao SO através de uma *System Call* (ex: read(), open()). A CPU muda para modo Kernel, executa a ação de forma segura e devolve o resultado.

Controlo de Acessos

Define quem pode fazer o quê:

- **DAC (Discricionário):** O dono do ficheiro decide (ex: permissões no Linux/Windows).
- **RBAC (Baseado em Papéis):** Permissões baseadas na função da pessoa (ex: "Gerente" pode ver tudo, "Caixa" só vê vendas).
- **ABAC (Baseado em Atributos):** Permissões dinâmicas (ex: "Pode aceder se for Gerente E se estiver no escritório").

📝 9. Explicação Prática dos Exercícios de Exame

Aqui estão as explicações passo a passo para os exercícios práticos mencionados no documento.

A. Cálculo de Endereço Físico (MMU) [Questão III & Exercício Extra]

O Problema: Converter um Endereço Virtual em Físico.

Dados Exemplo:

- Sistema de Memória: 128 KB (2^{17})
- Tamanho da Página: 16 KB (2^{14})
- Endereço Virtual: 25820

Passo a Passo:

1. **Calcular bits de Offset:** O tamanho da página é 2^{14}, logo os últimos 14 bits são o deslocamento (offset) dentro da página. Eles não mudam.
2. **Calcular bits da Página:** O sistema total é 2^{17}. $17 - 14 = 3$. Os **3 bits mais à esquerda** indicam o número da página.
3. **Converter Virtual para Binário:**
    - 25820 em binário = 110 0100 1101 1100.
    - Ajustar para 17 bits (adicionar zeros à esquerda): 001 10010011011100.
4. **Isolar o ID da Página:** Os 3 primeiros bits são 001. Isto é a **Página 1**.
5. **Consultar a Tabela:** Imagine que a tabela diz que a Página 1 está no Frame 5 (Binário 101).
6. **Substituir:** Trocamos o ID da página (001) pelo ID do Frame (101).
    - Novo Binário: 101 10010011011100.
7. **Converter para Decimal:** O valor binário 10110010011011100 corresponde a **91356**. Este é o endereço físico.

B. Algoritmo Buddy (Alocação de Memória) [Questão V]

O Problema: Gerir 1MB de memória dividindo sempre por 2.

Lógica:

- Se chega um pedido de 256KB e temos 1MB livre:
    1. Divide 1MB -> 512KB + 512KB.
    2. Divide o primeiro 512KB -> 256KB + 256KB.
    3. Aloca um dos 256KB.
- **Fusão:** Quando se liberta memória, verifica-se o "Buddy" (o bloco vizinho que resultou da divisão original). Se ambos estiverem livres, fundem-se novamente num bloco maior (256+256 volta a ser 512).

C. Escalonamento (Round-Robin vs SJF) [Questão IV]

Dados: Processos com diferentes tempos de chegada e duração.

1. **SJF (Shortest Job First):**
    - Olha para quem está pronto. Escolhe o que tem menor duração total.
    - Vantagem: Dá o menor tempo médio de espera.

2. **Round-Robin (Quantum = 0.2):**
    - Pega no primeiro da fila. Deixa correr 0.2 segundos.
    - Se não acabou, o processo vai para o fim da fila e entra o próximo.
    - Causa mais trocas de contexto (overhead), mas nenhum processo fica bloqueado à espera muito tempo (mais justo).

D. Algoritmo do Banqueiro (Evitação de Deadlock)

Objetivo: Saber se o sistema está num "Estado Seguro".

Lógica:

1. Calcula-se a matriz **Need** (Necessidade) = **Max** (O máximo que o processo precisa) - **Allocation** (O que já tem).
2. Verifica-se se existe algum processo cuja necessidade (**Need**) seja menor ou igual aos recursos disponíveis (**Available**).
3. Se sim, simula-se que esse processo executa, termina e devolve todos os seus recursos ao **Available**.
4. Repete-se para os outros. Se conseguirmos executar todos até ao fim, o sistema é **Seguro** e o Deadlock é evitado. Sequência segura ex: P1 -> P3 -> P4 -> P0 -> P2.